Αδιέξοδα (Deadlocks)

Περίληψη

- Αδιέξοδα (deadlocks)
- Τύποι πόρων (preemptable – non preemptable)
- Μοντελοποίηση αδιεξόδων
- Στρατηγικές
  - Στρουθοκαμηλισμός (ostrich algorithm)
  - Ανίχνευση και αποκατάσταση (detection and recovery)
  - Αποφυγή αδιεξόδων (deadlock avoidance)
  - Πρόληψη (deadlock prevention)

Παράδειγμα

- Υποθέστε πως δύο διεργασίες θέλουν να κάνουν τα εξής
  - Η `process1()` θα διαβάζει δεδομένα από το CD και θα τα αποθηκεύει σε μαγνητική ταινία
  - Η `process2()` θα διαβάζει δεδομένα από τη μαγνητική ταινία και θα τα αποθηκεύει στο CD.
- Για να αποφευχθούν οι συνθήκες συναγωνισμού θα πρέπει να χρησιμοποιηθεί αμοιβαίος αποκλεισμός έτσι που μόνο μια διεργασία να έχει πρόσβαση σε κάθε πόρο.
- Πώς μπορεί να επιτευχθεί;

Πρόσβαση σε Συσκευές Ε/Ε

```
semaphore cd=1, tp=1;
void process1(){
    while(true){
      down(&cd);
      down(&tp);
      in= readCD();
      writeTp(in);
      up(&cd);
      up(&tp);
    }
}
```

```
void process2(){
    while(true){
      down(&tp);
      down(&cd);
      in= readTp();
      writeCD(in);
      up(&cd);
      up(&tp);
    }
}
```

- **Αδιέξοδο (Deadlock)!!!**

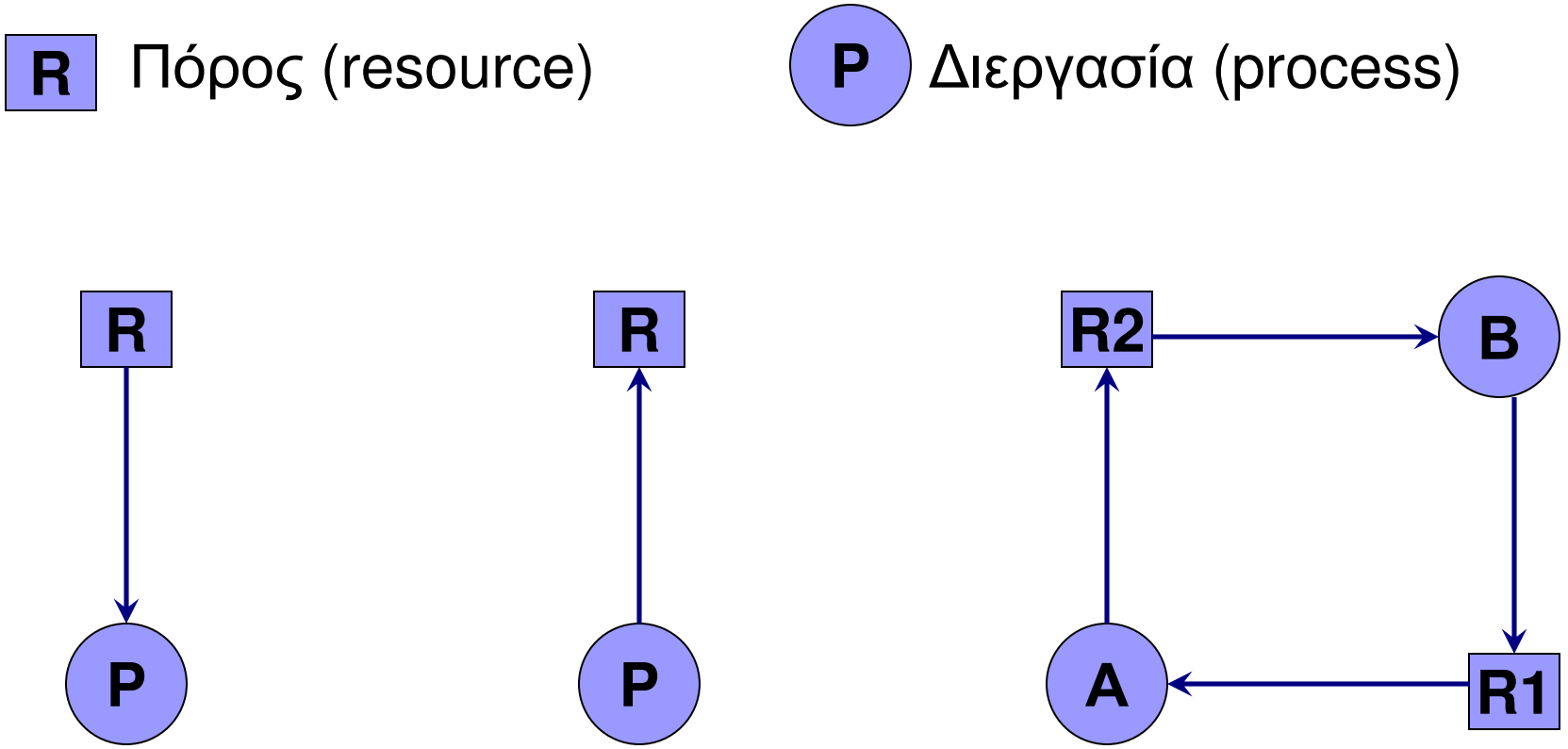
Τύποι Πόρων

- Preemptable resources
  - Πόροι οι οποίοι **μπορούν** να ελευθερωθούν προσωρινά (πριν τελειώσει μια διεργασία να τους χρησιμοποιεί) έτσι που να δώσουν την ευκαιρία σε άλλη διεργασία να συνεχίσει
    - Η κύρια μνήμη μπορεί να χρησιμοποιηθεί σαν preemptable. Γιατί;
- Non-Preemptable resources
  - Πόροι οι οποίοι **δεν μπορούν** να ελευθερωθούν προσωρινά (πριν τελειώσει μια διεργασία να τους χρησιμοποιεί).
    - Ο εκτυπωτής δεν μπορεί να χρησιμοποιηθεί σαν preemptable. Γιατί;
- Οι non preemptable πόροι είναι υποψήφιοι να προκαλέσουν κάποιο αδιέξοδο.

Αδιέξοδο

- Ένα σύνολο διεργασιών βρίσκονται σε αδιέξοδο εάν η κάθε μια περιμένει για κάποιο γεγονός το οποίο μόνο μια διεργασία του συνόλου μπορεί να προκαλέσει.
  - Η μια περιμένει την άλλη και γι' αυτό τίποτα δεν μπορεί να συμβεί
- Για τη δημιουργία αδιεξόδου **όλες** οι πιο κάτω συνθήκες πρέπει να ισχύουν
  - Συνθήκες αμοιβαίου αποκλεισμού
  - Συνθήκες κράτησης και αναμονής (hold and wait).
    - Διεργασία η οποία κρατεί κάποιους πόρους ζητά κάποιους επιπρόσθετους
  - Συνθήκες non-preemption
  - Συνθήκες κυκλικής αναμονής (circular wait conditions)
    - Κάθε διεργασία περιμένει κάποιο πόρο τον οποίο κρατά μια άλλη.

Μοντελοποίηση Αδιεξόδων

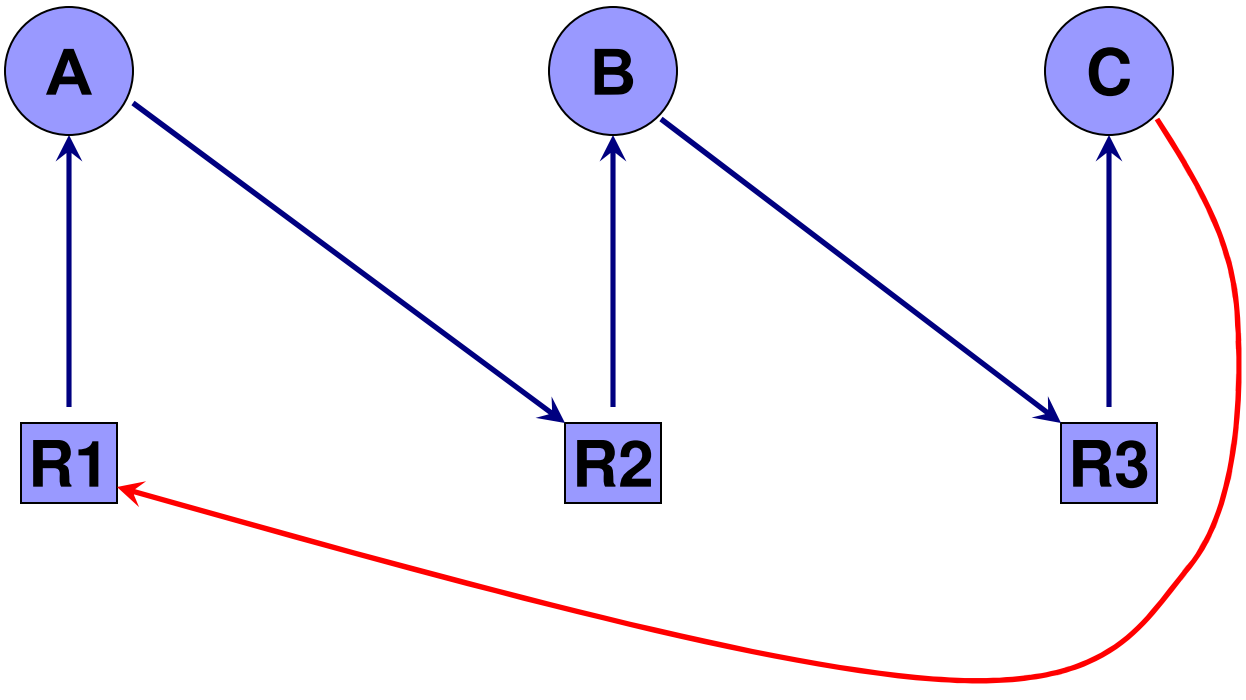


Ο πόρος R κρατείται από τη διεργασία P

Η διεργασία P περιμένει για τον πόρο R

Αδιέξοδο

Παράδειγμα



- A ζητά τον R1
- B ζητά τον R2
- C ζητά τον R3
- A ζητά τον R2
- B ζητά τον R3
- C ζητά τον R1

Αδιέξοδο

Στρατηγικές

- Στρουθοκαμηλισμός (ostrich algorithm)
- Ανίχνευση και αποκατάσταση (detection and recovery)
- Αποφυγή αδιεξόδων (deadlock avoidance)
- Πρόληψη (deadlock prevention)

Στρουθοκαμηλισμός (ostrich algorithm)

- Απλά αγνοούμε το πρόβλημα (reboot)!
- Εάν αδιέξοδα συμβαίνουν πολύ σπάνια και δεν τίθεται θέμα ασφάλειας τότε αυτή μπορεί να είναι «καλή» λύση.
- Η λύσεις στο πρόβλημα (με μία από τις άλλες στρατηγικές) είναι δαπανηρή αφού μειώνεται η απόδοση του συστήματος.
  - Ό συνεχής έλεγχος για πιθανά αδιέξοδα καταναλώνει μέρος της υπολογιστικής ισχύος του επεξεργαστή καθώς και άλλους πόρους του συστήματος.
- Το Unix και Windows ακολουθούν αυτή τη στρατηγική
- Τι θα συμβεί εάν η εφαρμογή είναι κρίσιμη;

Ανίχνευση και Αποκατάσταση (Detection and Recovery)

- Αλγόριθμος για ανίχνεύση του Αδιεξόδου
- Όταν ανιχνευθεί αδιέξοδό τότε λαμβάνονται κάποια μέτρα για αποκατάσταση του αδιεξόδου
  - Αποκατάσταση χρησιμοποιώντας preemption
  - Αποκατάσταση χρησιμοποιώντας rollback
  - «Αποκατάσταση» τερματίζοντας κάποιες από τις διεργασίες
    - Στην πράξη καμία μέθοδος δεν είναι ιδιαίτερα ικανοποιητική.
- Πρόβλημα: πόσο συχνά τρέχουμε τον αλγόριθμο ανίχνευσης του αδιεξόδου;

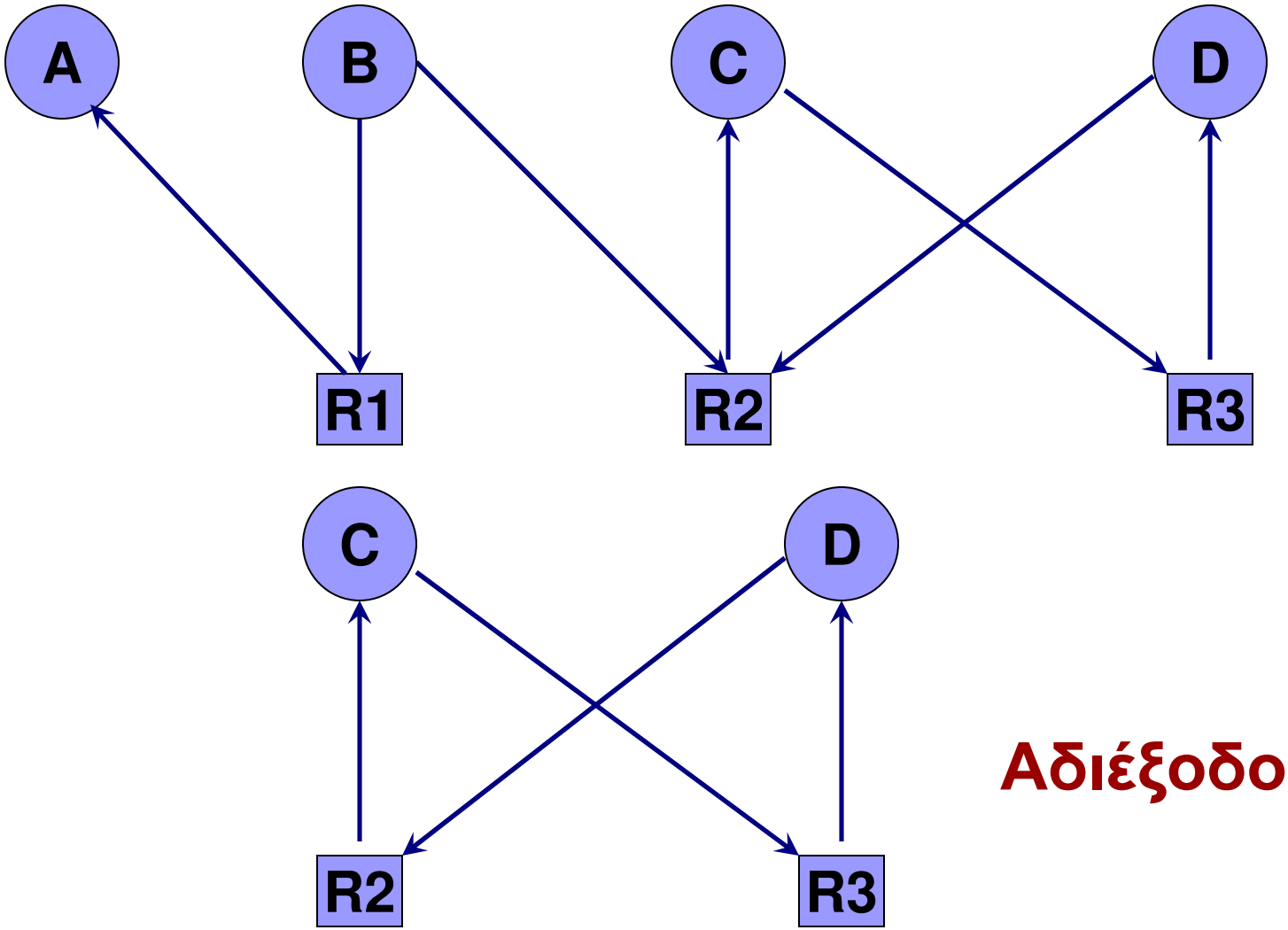
Μέθοδοι Αποκατάστασης

- Αποκατάσταση χρησιμοποιώντας **preemption**
  - Μια διεργασία διακόπτεται υποχρεωτικά
  - Ελευθερώνει τους πόρους που κρατεί για να χρησιμοποιηθούν από άλλες διεργασίες
- Αποκατάσταση χρησιμοποιώντας **rollback**
  - Στον κώδικα υπάρχουν κάποια ελεγχόμενα σημεία (checkpoints) στα οποία αποθηκεύεται όλη η κατάσταση της διεργασίας (καταχωρητές, πόροι κλπ)
  - Μόλις ανιχνευθεί αδιέξοδο, η διεργασία διακόπτεται και επιστρέφει σε κάποιο από τα προηγούμενα σημεία ελέγχου ελευθερώνοντας κάποιους πόρους για να χρησιμοποιηθούν από άλλες διεργασίες.
- «Αποκατάσταση» τερματίζοντας (**kill**) κάποιες από τις διεργασίες
  - Απλά τερματίζουμε κάποιες διεργασίες για να ελευθερώσουν τους πόρους τους έτσι που να μπορούν οι υπόλοιπες να συνεχίσουν.

Αλγόριθμος Ανίχνευσης Αδιεξόδου για «απλούς» πόρους

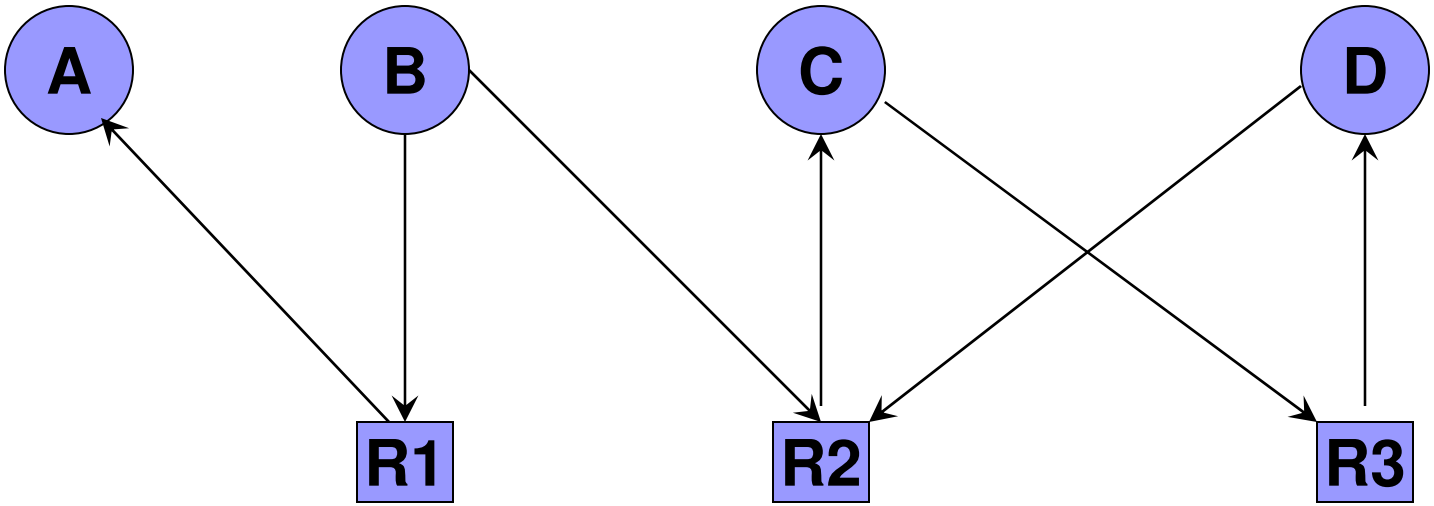
- Απλοί πόροι: το σύστημα διαθέτει μόνο ένα πόρο από κάθε είδος.
- Χρησιμοποιώντας το προαναφερθέν γραφικό μοντέλο είναι εύκολο να εντοπίσουμε τα αδιέξοδα: κοιτάζουμε κατά πόσο υπάρχει κάποια «κλειστή αλυσίδα»
- Υπάρχει κάποιος αλγόριθμος που να μας βοηθήσει να εντοπίσουμε κάποια κλειστή αλυσίδα σε ένα διάγραμμα και που να μπορεί να υλοποιηθεί αυτόματα (σαν ένα πρόγραμμα);

Παράδειγμα



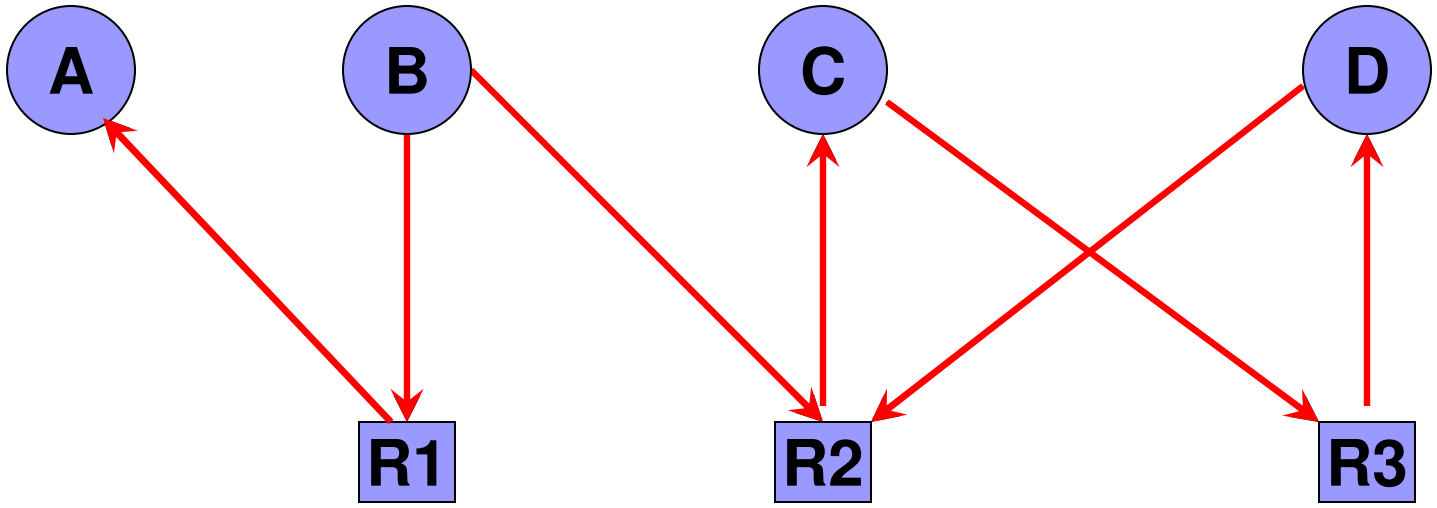
Αδιέξοδο

Αλγόριθμος Ανίχνευσης Αδιεξόδου



- Ξεκινούμε από κάποιο κόμβο και ακολουθούμε τα βέλη.
- Κάθε κόμβος που επισκεπτόμαστε σημειώνεται σε μία λίστα.
- Εάν εμφανιστεί ο ίδιος κόμβος στη λίστα δύο φορές τότε υπάρχει κλειστή αλυσίδα.
- Ο πιο πάνω αλγόριθμος πρέπει να ξεκινήσει από όλους τους δυνατούς κόμβους.
- Πρέπει να υπάρχει τρόπος να μην ακολουθούμε το ίδιο μονοπάτι δύο φορές.

Αλγόριθμος Ανίχνευσης Αδιεξόδου



Λίστα L= **A**

Λίστα L= **B R1 A R2 C R3 D R2** **Αδιέξοδο**

Πρόληψη Αδιεξόδων

- Για τη δημιουργία αδιεξόδου **όλες** οι πιο κάτω συνθήκες πρέπει να ισχύουν
  - Συνθήκες αμοιβαίου αποκλεισμού
  - Συνθήκες κράτησης και αναμονής (hold and wait).
    - Διεργασία η οποία κρατεί κάποιους πόρους ζητά κάποιους επιπρόσθετους
  - Συνθήκες non-preemption
  - Συνθήκες κυκλικής αναμονής (circular wait conditions)
    - Κάθε διεργασία περιμένει κάποιο πόρο τον οποίο κρατά μια άλλη.
- Κατά την πρόληψη αδιεξόδων γίνεται προσπάθεια να «σπάσει» τουλάχιστον μια από τις πιο πάνω συνθήκες έτσι που να αποφευχθεί το αδιέξοδο.

Πρόληψη Αδιεξόδων

- Συνθήκες αμοιβαίου αποκλεισμού
  - Οι συνθήκες αμοιβαίου αποκλεισμού είναι αναγκαίες για την ορθή λειτουργία ορισμένων συσκευών
    - Π.χ., Όταν μια διεργασία ξεκινήσει να τυπώνει δεν μπορεί μια άλλη ταυτόχρονα να χρησιμοποιεί τον εκτυπωτή!
- Συνθήκες κράτησης και αναμονής (hold and wait).
  - Πιθανή λύση είναι να υποχρεώνουμε όλες τις διεργασίες να ζητούν όλους τους πόρους που θα χρειαστούν και να μην ξεκινούν αν δεν τους κατανεμηθούν όλοι.
    - Πολλές διεργασίες δεν ξέρουν από πριν τους πόρους που θα χρειαστούν!
    - Τέτοια λύση δεν είναι ιδιαίτερα αποδοτική.

Πρόληψη Αδιεξόδων

- Συνθήκες non-preemption
  - Απλά η αφαίρεση κάποιων πόρων από μια διεργασία έτσι που να τη χρησιμοποιήσει κάποια άλλη δεν είναι εφικτό ή και αν είναι εφικτό είναι μάλλον δύσκολο.
- Συνθήκες κυκλικής αναμονής (circular wait conditions)
  - Να καθοριστεί μια μοναδική και καθολική σειρά την οποία να ακολουθούν όλες οι διεργασίες
    - Π.χ., (1) Εκτυπωτής, (2) scanner, (3) plotter (4)…
  - Εναλλακτικά, μια διεργασία δεν μπορεί να ζητήσει πόρους με αριθμό μικρότερο από αυτούς πού έχει ήδη.
  - Υπάρχουν τόσα πολλά είδη πόρων που είναι δύσκολο (αδύνατο) να βρεθεί μια σειρά που να ικανοποιεί όλες τις διεργασίες.

Πιο πολλά

Στο μάθημα: Λειτουργικά Συστήματα ☺